1 Lecture 15 (Oct 15) - Dynamic Programming on Trees: Woodcutting, Maximum Independent Set

So far, dynamic programming has read one or two arrays from one end to the other. Today the recursion has a different shape: it builds a **tree**. This is the last lecture on dynamic programming for its own sake.

> **Key idea** **Make one decision, and the rest splits into independent pieces:** Solve each piece recursively and add up the costs. Woodcutting splits a plank into two subplanks. Independent set splits a tree into its subtrees.

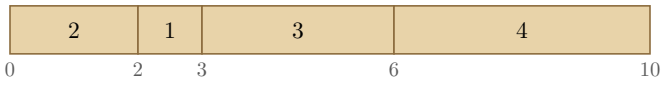
1.1 Woodcutting

1.1.1 The problem

A plank of wood has $n-1$ marks on it. It must be cut at every mark, and only at the marks, into n final pieces. Only one company owns a saw big enough. It does **one cut at a time** and charges the **length of the plank it is cutting**. Each cut gives two smaller planks, which you hand back for more cuts.

Input: an array Len[1..n] of the final piece lengths, in order from left to right. **Output:** the minimum total cost to make all the cuts.

Example: A plank of length 10 with marks at 2, 3 and 6, so $n = 4$ and Len $= [2, 1, 3, 4]$:



The first cut always costs \$10, whichever mark it uses. After that, the order matters:

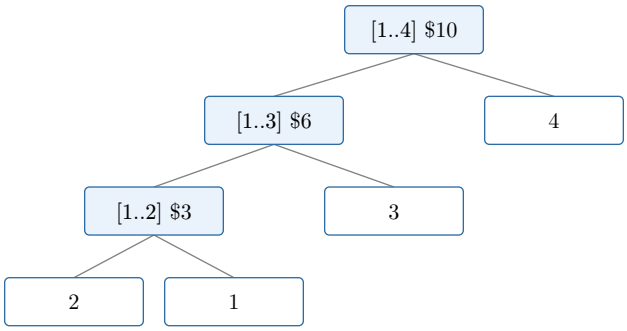
Strategy	Cuts, in order (plank length paid)	Total
Left to right	at 2 (10), at 3 (8), at 6 (7)	\$25
Middle first	at 3 (10), then at 2 (3) and at 6 (7)	\$20
Right to left	at 6 (10), at 3 (6), at 2 (3)	**\$19**

> **Watch out** **The most balanced cut is not always best:** Right to left wins here: the long piece of length 4 is cut off first, so you stop paying for it early.

1.1.2 Part 1: Formulate recursively

Decision: where to make the first cut. It costs the length of the whole plank. It leaves a left subplank and a right subplank, and these are **independent** problems: how you cut one does not change the cost of the other.

The cuts form a **binary tree**. The root is the whole plank, and its two children are the two subplanks. Choosing the first cut means choosing the root. The optimal strategy for the example:



Internal nodes: the pieces on that plank and the cost of its cut. Leaves: final piece lengths.

Every subplank holds a **contiguous** run of final pieces, so a pair of indices names it.

Subproblem: For $1 \le i \le k \le n$:

$$\text{Cutting}(i, k) = \text{minimum cost to cut the plank made of pieces Len}[i..k] \text{ into those pieces}$$

Recurrence: Guess the index r of the first piece in the right subplank. The right subplank must not be empty, and neither may the left one, so $i < r \le k$:

$$\text{Cutting}(i,k) = \begin{cases} 0 & \text{if } i = k \\ \sum_{j=i}^{k} \text{Len}[j] + \min_{i<r\le k} \left(\text{Cutting}(i, r-1) + \text{Cutting}(r, k)\right) & \text{otherwise} \end{cases}$$

The sum is the cost of the first cut (the length of the whole subplank). The min tries every place for that cut. A single piece ($i = k$) needs no cuts.

Original problem: Cutting$(1, n)$.

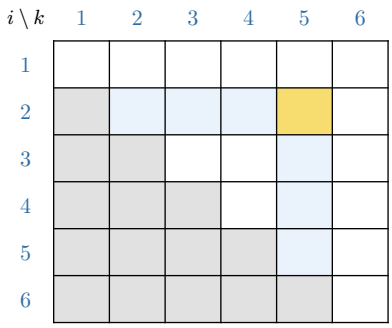
> **Key idea** **Try every root. Don't try to be clever:** You don't know which cut goes first, so try all of them, and let the recursion handle each side.

1.1.3 Part 2: Build solutions bottom-up

Subproblems: Cutting(i, k) for all $1 \le i \le k \le n$.

Data structure: a 2D array Cutting$[1..n, 1..n]$. Only the entries with $i \le k$ are used.

Dependencies: Entry (i, k) reads Cutting$(i, r-1)$ for $i \le r-1 < k$: the **same row, to the left**. It also reads Cutting(r, k) for $i < r \le k$: the **same column, below**. For $n = 6$ and entry $(2, 5)$:



Rows are i, columns are k. Yellow: the entry being filled. Blue: the entries it reads. Gray: unused ($i > k$).

Evaluation order: Any order that fills the row to the left and the column below first. Three choices work:

Order	Loops	Verdict
Row by row	i from n **down** to 1; inside, k from i **up** to n	Good
Column by column	k from 1 **up** to n; inside, i from k **down** to 1	Good
Diagonal by diagonal	Short subplanks first ($k - i = 0, 1, 2, ...$)	Correct, but the indices are easy to get wrong

> **Watch out** **The two loops go in opposite directions**: one counts up, one counts down. Read each direction off the dependency drawing. Don't assume both go the same way.

Running time: Each entry adds up to n lengths and tries up to n values of r, so $O(n)$ time per entry. (Erickson precomputes the sums, but it doesn't change the answer here.) There are $O(n^2)$ entries:

$$O(n^2) \text{ subproblems} \times O(n) \text{ each} = \boxed{O(n^3)}$$

On homework, you can stop here. The pseudocode is just nested loops, and it is no easier to read than the steps above. For reference (Erickson's book names the array `MinCost`):

```
Woodcut(Len[1..n]):
  for i ← n down to 1
    Cutting[i, i] ← 0
    for k ← i + 1 to n
      total ← Len[i] + Len[i + 1] + ... + Len[k]
      best ← ∞
      for r ← i + 1 to k
        best ← min(best, Cutting[i, r − 1] + Cutting[r, k])
```

```
    Cutting[i, k] ← total + best
return Cutting[1, n]
```

The example, filled in: Len $= [2, 1, 3, 4]$, row by row from the bottom:

$i \setminus k$	1	2	3	4
1	0	3	9	19
2		0	4	12
3			0	7
4				0

For example, $\text{Cutting}(1, 4) = 10 + \min(0 + 12, 3 + 7, 9 + 0) = 19$. The best first cut is $r = 4$: cut off the last piece, as in the right-to-left strategy.

1.2 Maximum Independent Set in Trees

1.2.1 The problem

Let $G = (V, E)$ be a graph. An **independent set** is a subset $S \subseteq V$ such that **no edge has both endpoints in S**. The goal: find the largest one.

In a general graph this is **NP-hard** (later in the semester). There is probably no polynomial-time algorithm, not even an $O(n^{100})$ one. So restrict the input to a **rooted tree** T with n nodes and root r. Nodes may have any number of children: it is not necessarily a binary tree. Rooted trees are recursive: a root, plus a rooted subtree for each child.

1.2.2 Part 1: Formulate recursively

Decision: is the root in the set or not?

- **Root not in the set:** Its children are all allowed. There are no edges between different subtrees, so take the best independent set of each child's subtree and combine them.
- **Root in the set:** Every child shares an edge with the root, so no child is allowed. The children's subtrees below them are free: take the best set in each **grandchild's** subtree, plus the root.

The recursive calls are always on subtrees. Name a subtree by its root, as with binary search trees in CS 225.

Subproblem: For each node v:

$$\text{MIS}(v) = \text{the size of the largest independent set in the subtree rooted at } v$$

Notation: $w \downarrow v$ means "w is a child of v".

Recurrence:

$$\mathrm{MIS}(v) = \max\left\{ \sum_{w \downarrow v} \mathrm{MIS}(w), \quad 1 + \sum_{w \downarrow v} \sum_{x \downarrow w} \mathrm{MIS}(x) \right\}$$

No separate base case is needed. At a leaf both sums are empty, so $\mathrm{MIS}(\text{leaf}) = \max\{0, 1\} = 1$.

Original problem: $\mathrm{MIS}(r)$.

1.2.3 Part 2: Build solutions bottom-up

Subproblems: one per node v.

Data structure: The subproblems are named by nodes, not integers, so an array no longer fits. Store each answer in a **field on the node**: v.MIS. (Same idea as a distance field on each vertex in Dijkstra's algorithm.)

Dependencies: v reads its children and grandchildren: nodes **below** v.

Evaluation order: postorder. It visits all of a node's descendants before the node.

> Key idea **"Evaluate in postorder" is a complete answer:** Computing a postorder is a black-box algorithm you may use. Compute it, then loop over the nodes in that order. The same answer works for directed acyclic graphs next week.

Running time: The usual (number of subproblems) $\times$ (time per subproblem) gives a bound that is far too big. The time per node varies a lot: one node may have $n/2$ children while most have none. Max time $\times$ count gives $O(n^2)$.

Instead, **add up the work and charge it to the node being read**. Node v costs $O(1 + \#\text{children} + \#\text{grandchildren})$. Each node is

- a child of at most one node, so it is read as a child at most once in total;
- a grandchild of at most one node, so it is read as a grandchild at most once in total.

So all the reads add up to $O(n)$, and the total is

$$O(n) \text{ for the nodes} + O(n) \text{ child reads} + O(n) \text{ grandchild reads} = \boxed{O(n)}$$

This is only an analysis trick: the algorithm does nothing extra. (Another view: there is one child read per edge, and a tree has $n - 1$ edges.)

> Watch out **For tree and graph DP, don't multiply the maximum time per subproblem by the number of subproblems.** Add the work over all nodes, and charge each read to the node (or edge) it touches.

1.2.4 A cleaner version: two subproblems

Split the subproblem in two, by whether v is in the set:

$$\text{MISyes}(v) = \text{largest independent set in } v\text{'s subtree that must contain } v$$
$$\text{MISno}(v) = \text{largest independent set in } v\text{'s subtree that must not contain } v$$

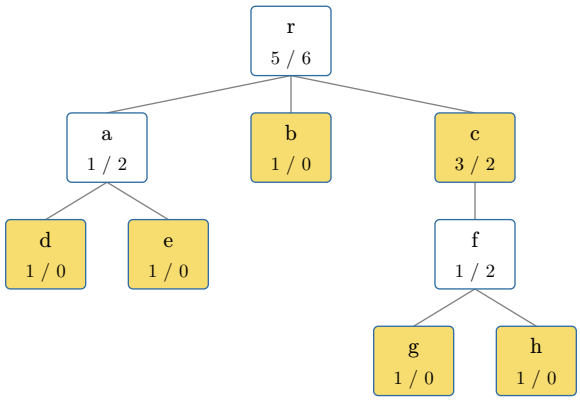
If v is in, no child is: there is no decision. If v is out, each child is free: take the better of in and out.

$$\text{MISyes}(v) = 1 + \sum_{w \downarrow v} \text{MISno}(w)$$
$$\text{MISno}(v) = \sum_{w \downarrow v} \max\{\text{MISyes}(w), \text{MISno}(w)\}$$

The two recurrences call each other, but there are no grandchildren sums. The answer is $\max\{\text{MISyes}(r), \text{MISno}(r)\}$. Store two fields on each node. Each node still depends only on its children, so evaluate in postorder: $O(n)$ time.

```
TreeMIS(T):
  for each node v in postorder
    v.yes ← 1
    v.no ← 0
    for each child w of v
      v.yes ← v.yes + w.no
      v.no ← v.no + max(w.yes, w.no)
  return max(r.yes, r.no)
```

Example: Each node shows MISyes / MISno. Yellow nodes form a maximum independent set of size $\max\{5, 6\} = 6$:



For example, $\text{MISyes}(c) = 1 + \text{MISno}(f) = 3$, and $\text{MISno}(r) = 2 + 1 + 3 = 6$ (the better value of a, b and c). The set: leave out r, a and f; take b, c, d, e, g, h.

1.3 Summary

Step	**Woodcutting**	**Tree MIS**
Decision	Where the first cut goes	Is the root in the set?
Subproblem	Cutting(i, k): pieces $i..k$	MIS(v): subtree at v
Original problem	Cutting$(1, n)$	MIS(r)
Data structure	2D array Cutting$[1..n, 1..n]$	Field v.MIS on each node
Dependencies	Same row to the left, same column below	Children and grandchildren
Evaluation order	i down from n; k up from i	Postorder
Running time	$O(n^2) \times O(n) = O(n^3)$	$O(n)$, by charging reads to nodes